Pagination And Sorting

Pagination :-

Pagination is a technique used to divide large datasets into smaller, manageable chunks (pages) to improve performance and reduce memory usage when retrieving data from a database.

In Spring Boot (Spring Data JPA), pagination is implemented using:

- **Page**
- **Pageable**
- **PageRequest**

It prevents loading all records at once and improves API response time.

Why Pagination?

Without pagination:- SELECT * FROM users
If you have 10 lakh records → your API will crash or become slow.
With pagination:- SELECT * FROM users LIMIT 10 OFFSET 20;
You fetch data in chunks (pages).

Spring Data JPA Built-in Support
Spring provides:

- Page
- Pageable
- PageRequest
- Sort

No manual SQL needed.

Step-by-Step Implementation

1.Entity

```
@Entity
@Table(name=”users”)
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class User {

   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private String name;
   private String email;
   private Integer age;
   private String password;
}
```

2. DTO Class

```
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public  class UserRequestDTO{
        private String name;
        private String email;
        private Integer age;
        private String password;
}
```

```
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public  class UserResponseDTO{
        private String name;
        private String email;
        private Integer age;
}
```

3. Repository

Just extend JpaRepository

```
public interface UserRepository extends JpaRepository<User, Long> {
}
```

4.Service

```
Public interface UserService {
    Public List< UserResponseDTO > fetchAllUser( Integer page, Integer
    size , String sortBy ,String direction);
}
```

5.ServiceImpl

```
@Service
@RequiredArgsConstructor
@Slf4j
@Transactional
public class UserServiceImpl implements UserService{

    private final UserRepository userRepository;
    private final ModelMapper modelMapper;
```

```
    public List< UserResponseDTO > fetchAllUser( Integer page, Integer size ,
    String sortBy ,String direction){
    long startTime = System.currentTimeMillis();
    log.info("Fetching users - page: {}, size: {}", page, size);

    Sort sort= direction.equalsIgnoreCase(“asc”)? Sort.by(sortBy).ascending():
                Sort.by(sortBy).descending():
    Pageable pageable= PageRequest.of(page,size,sort);
    Page<User> userPage= userRepository.findAll(pageable);
    List< UserResponseDTO > allUser= userPage.stream().map((user)->
    this.modelMapper(user, UserResponseDTO));

    log.info("Fetched {} users out of total {} records",
    userPage.getNumberOfElements() , userPage.getTotalElements());

    long endTime = System.currentTimeMillis();
    log.debug("Fetch users execution time: {} ms", (endTime - startTime));

    return  allUser;
    }
}
```

6.Controller

```
@RestController
@RequestMapping( "/api/v1/users")
@RequiredArgsContructor
 public  class UserController {
    private final UserService userService ;
    @GetMapping(“/get”)
    Public ResponseEntity<List< UserResponseDTO >> getAllUser(@Valid
        @RequestParam(name=”page” , defaultValue=”0”) Integer page,
        @RequestParam(name=”size” , defaultValue=”0”) Integer size,
```

```
        @RequestParam(name=”id” , defaultValue=”0”) String sortby,
        @RequestParam(name=”asc” , defaultValue=”0”) String direction ){

        List< UserResponseDTO >  userList= userService.fetchAllUser(page ,
        size , sortBy , direction);
        log.info("API call received: GET
        /api/v1/users?page={}&size={}&sortBy={}&direction={}", page, size);
        return new ResponseEntity<>(userList , HttpStatus.ok);
        }
}
```

Hit this url in Postman: –
http://localhost:8080/api/v1 /users?page=0&size=5&sortBy=age&direction=desc

Sorting Only (Without Pagination)

```
List<User> users = userRepository.findAll(Sort.by("name").descending());
```

Custom Query with Pagination

```
@Query("SELECT u FROM User u WHERE u.age > :age")
Page<User> findUsersOlderThan(@Param("age") int age, Pageable pageable);
```

Usage:

```
Pageable pageable = PageRequest.of(0, 5, Sort.by("name"));
userRepository.findUsersOlderThan(25, pageable);
```

Multi-column Sorting

```
Sort sort = Sort.by("age").descending()
        .and(Sort.by("name").ascending());

Pageable pageable = PageRequest.of(0, 10, sort);
```